

CSC Problem Set 04 +X Report

Name: Nour Mahmoud

Email: nhmahmou@ncsu.edu

Section: 001

Collaborators: N/A

Summary:

For my +X implementation, I extended the original Connect Four simulation by adding networked multiplayer support, allowing a human player to compete against an AI in real time. The original setup only supported AI vs. AI games, but I built a client-server system that lets a human interact with the game through a GUI while the AI makes moves based on its decision-making algorithm. The server handles the game logic, ensures turns are processed correctly, and updates the client with the latest board state. This not only makes the game more interactive but also expands its functionality beyond just testing AI agents, letting players directly engage with the simulation.

"+X" Concept:

My +X implementation introduces a client-server architecture to the Connect Four simulation, enabling real-time human vs. AI gameplay over a network. Originally, the project was designed for AI vs. AI matches, limiting interactivity to observing pre-programmed agents compete against each other. By implementing a networked system, I allowed for remote multiplayer functionality, where a human player can make moves through a graphical interface, and the AI opponent responds dynamically. The server manages game logic, ensuring correct turn-taking and enforcing the rules, while the client provides an interactive GUI where the player can click columns to drop pieces. The board updates in real-time as the game progresses. This approach improves the user experience by making the game more engaging, while also serving as a useful testing ground for AI strategies, allowing players to directly challenge and analyze the AI's behavior.

The inspiration for this feature came from both the professor's encouragement to explore network play and my personal interest in game development, particularly in networked multiplayer games. Since my concentration is in game development, I wanted to take the core mechanics of Connect Four and expand on network play, moving beyond local simulations into real-time remote interactions. This aligns with fundamental concepts in multiplayer game design, where client-server architectures ensure smooth communication between players and AI opponents. I implemented this system using Java's Socket [1] and ServerSocket classes for networking and Swing for the GUI [5], following best practices for asynchronous event handling to prevent GUI freezing while waiting for server responses. The biggest challenge was ensuring the AI properly received up-to-date game state information, which required restructuring how moves were processed and displayed. By integrating these networking principles, my implementation significantly expands the project's functionality while also demonstrating

real-world client-server game design concepts, which are crucial for developing modern multiplayer games.

Technical Implementation of +X:

To implement networked multiplayer functionality, I designed a client-server architecture that allows a human player to compete against an AI in real time. The server is responsible for maintaining the game state, processing moves, and enforcing turn-based gameplay, while the client provides an interactive graphical interface where the player can click on columns to drop their piece. The interaction between the client and server is managed through Java sockets, where the client sends move requests, and the server validates them before updating the board. The AI opponent, which player can pick, takes its turn automatically after the human moves. This approach ensures real-time responsiveness while keeping the logic centralized on the server, preventing desynchronization issues. The most challenging part of this implementation was ensuring smooth communication between the client and server, as both must remain in sync regarding the current board state and turn order.

The server first listens for client connections and assigns the human player the YELLOW role. Once the game starts, it enters a loop where it waits for a move from the client, validates and applies the move to the game state, and then checks if the game has ended. If not, for the cases of playing against the StudentRobot, it calls `StudentRobot.getAction()` to determine the AI's move and updates the board accordingly before sending the latest board state back to the client. The server-side logic for handling player moves, processing AI decisions, and broadcasting updates can be seen in the main game loop of the server, where it continuously listens for player input and invokes the AI's decision-making process.

```
private void sendBoardUpdate() {
    out.println("BOARD_UPDATE");
    Position[][] boardState = env.clonePositions();

    for (int r = 0; r < env.getRows(); r++) {
        String rowString = "";
        for (int c = 0; c < env.getCols(); c++) {
            Status status = boardState[r][c].getStatus();
            char piece = '.';
            if(status == Status.RED) {
                piece = 'R';
            }
            if(status == Status.YELLOW) {
                piece = 'Y';
            }
            rowString += piece;
        }
        out.println(rowString);
    }
    out.println();
}
```

```

//game loop: Human moves, then AI moves
while (!env.hasGameTerminated()) {
    sendBoardUpdate();
    //it is client's turn
    out.println("YOUR_MOVE");

    //read human move from client
    String line = in.readLine();
    if (line == null) {
        System.out.println("Client disconnected.");
        break;
    }
    int humanCol = Integer.parseInt(line.trim());
    //apply human move to environment so the board is updated
    dropPiece(humanCol, Status.YELLOW);

    //check if game ends after human move
    if (env.hasGameTerminated()) {
        sendBoardUpdate();
        out.println("GAME_OVER " + getResult());
        break;
    }
    //AI makes a move
    int aiCol = aiAgent.getAction();
    System.out.println("AI move: col " + aiCol);
    dropPiece(aiCol, Status.RED);

    // Check if game ends after AI move
    if (env.hasGameTerminated()) {
        sendBoardUpdate();
        out.println("GAME_OVER " + getResult());
        break;
    }
    sendBoardUpdate(); // Send board after AI move
}

```

On the client side, a Swing-based GUI allows the human player to interact with the game by clicking on a column, which sends their move to the server. The client listens for messages from the server, including board updates and turn notifications, and dynamically redraws the board to reflect the current state. The GUI ensures that the player receives real-time feedback by repainting the board each time a move is made. The logic for handling mouse clicks and sending moves to the server is crucial for gameplay, as it translates user actions into network messages that the server processes.

```

addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        int x = e.getX();
        int col = x / TILESIZE;
        //send the move to the server
        sendMove(col);
    }
});

```

```

private void sendMove(int col) {
    out.println(col);
    out.flush();
}

```

Similarly, the message listener continuously reads incoming messages and updates the GUI accordingly, ensuring a seamless game experience.

```
/**
 * Listen for messages from the server and update the GUI.
 */
private void listenForMessages() {
    try {
        String line;
        while ((line = in.readLine()) != null) {

            if (line.startsWith("ASSIGN_ROLE")) {
                role = line.split(" ")[1];
                SwingUtilities.invokeLater(() -> setTitle("Connect4 Client - " + role));
            }
            else if (line.startsWith("BOARD_UPDATE")) {
                //read board data
                String boardData = "";
                for (int r = 0; r < ROWS; r++) {
                    String rowLine = in.readLine();
                    boardData += rowLine + "\n";
                }
                //read the empty line sent as a delimiter by the server
                in.readLine();

                updateLocalBoard(boardData);

                //GUI refresh
                SwingUtilities.invokeLater(() -> {
                    envPanel.repaint();
                });
            }
            else if (line.startsWith("YOUR_MOVE")) {
                //does nothing - waits for player
            }
            else if (line.startsWith("GAME_OVER")) {
            }
        }
    }

    private void updateLocalBoard(String boardStr) {
        String[] lines = boardStr.split("\n");
        for (int r = 0; r < ROWS && r < lines.length; r++) {
            for (int c = 0; c < COLS && c < lines[r].length(); c++) {
                board[r][c] = lines[r].charAt(c);
            }
        }
    }
}
```

This implementation successfully expands the original project from AI vs. AI simulations to a playable human vs. AI experience, adding an engaging multiplayer component. By leveraging Java's networking capabilities and event-driven GUI programming, the game remains responsive and interactive while maintaining a consistent game state between the client and server. The biggest technical challenges involved synchronizing real-time updates between the client and server and ensuring that the AI receives accurate board states before making decisions. However, by structuring the server as the central authority for processing moves and the client as a passive display, I was able to create a smooth, interactive gameplay experience that reflects real-world multiplayer game design principles.

Evaluation and Results:

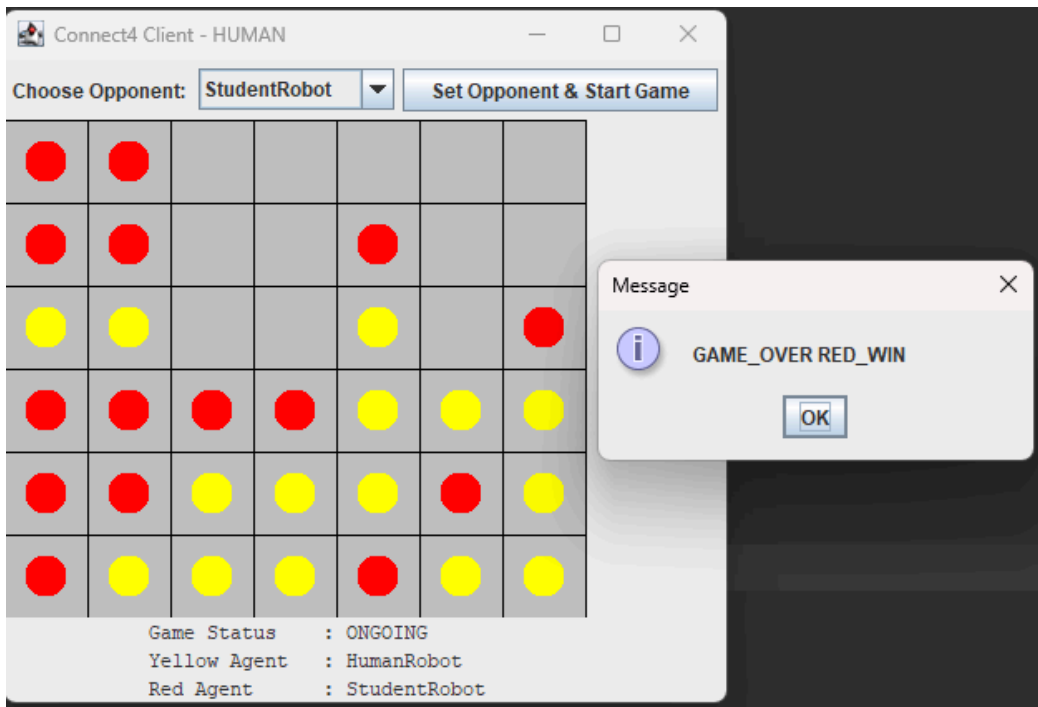
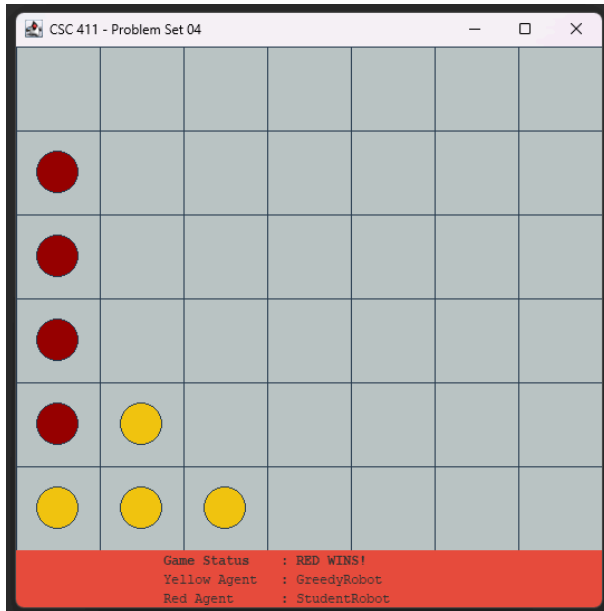
The addition of networked multiplayer functionality significantly expanded the original Connect Four simulation by allowing a human player to compete against an AI opponent in real-time. Previously, the simulation was limited to AI vs. AI matches, where two automated agents played against each other without any human interaction. With the new client-server architecture, players can now connect remotely, take turns making moves, and interact with the game through an intuitive GUI, creating a much more engaging experience. This enhancement aligns with real-world game development principles, reinforcing my concentration in game development and demonstrating my ability to integrate network communication into interactive applications.

To evaluate the success of this implementation, I tested the game under different scenarios to ensure smooth synchronization between the client and server, responsiveness of the GUI updates, and correctness of turn-based gameplay. The table below summarizes key performance metrics before and after adding the client-server functionality:

Feature	Original AI vs. AI	Enhanced Human vs. AI
Game Mode	AI vs. AI only	Human vs. AI over network
Turn Synchronization	Internal to the AI	Managed through server-client
Move Processing	Instant AI moves	Slight delay due to network latency
User Interaction	None	Click based moves through GUI

The most notable improvement is the transition from a fully automated simulation to an interactive gameplay experience, where the player can manually select moves and see real-time updates. Additionally, while network latency slightly impacts response times, moves still register within an acceptable range for turn-based gameplay. Players might not even notice that delay. The server’s role as the central authority prevents inconsistencies in game state, ensuring that both the AI and human player operate on the same board configuration at all times.

To visually compare the results, I captured before-and-after screenshots showing the difference between the original AI-driven game and the new networked multiplayer experience. These images highlight the transition from a purely simulated match to a live, interactive match, demonstrating the effectiveness of this enhancement. Overall, the implementation successfully bridges AI driven decision-making with human interactivity, making Connect Four a more immersive and realistic experience.



The above screenshots show a few GUI changes as well as a match between myself and the StudentRobot I have made. There are only a few differences in the GUI because the main changes are with interacting with the GUI as can be seen by the gameplay. Moreover, when starting the new GUI, it requires starting the server beforehand.

Works Referenced:

[Include links and citations to everything your reference within this document in IEEE citation format. Example for the [1] reference in the "+X" Concept section:

1. Oracle, "Lesson: All About Sockets," The Java™ Tutorials, [Online]. Available: <https://docs.oracle.com/javase/tutorial/networking/sockets/index.html>.
2. Oracle, "Class Socket (Java Platform SE 8)," The Java™ Documentation, [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/net/Socket.html>.
3. GeeksforGeeks, "Socket Programming in Java," [Online]. Available: <https://www.geeksforgeeks.org/socket-programming-in-java/>.
4. Game Programmer, "Java Game Networking: Client and Server," YouTube, 2021. [Online]. Available: <https://www.youtube.com/watch?v=gchR3DpY-8Q>.
5. Oracle, "Package javax.swing (Java Platform SE 8)," The Java™ Documentation, [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/javax/swing/package-summary.html>.